

Épreuve de Programmation Orientée Objet 2

Licence Informatique — Sujet d'entraînement A

Singleton, Enum et Strategy

Juin 2026

Durée : 2 heures. **Documents autorisés :** documentation des classes conteneur (livret distribué) et de l'interface `JAVA Iterator`.

Ce sujet est un sujet d'entraînement généré à partir du contenu du cours de POO2 (Iterator, Singleton/Enum, Strategy, Decorator, Streams, Command/MVC, Builder). Il n'a pas valeur de sujet officiel mais reprend le format et l'esprit des épreuves de M. Champesme.

Ce qui est évalué dans cette épreuve

Dans cette épreuve sont évaluées vos capacités à : - utiliser correctement le langage JAVA pour implémenter une classe ; - effectuer des choix corrects pour la visibilité des champs et des méthodes ; - respecter le contrat hérité des super-classes et des interfaces lors de la définition d'une nouvelle classe ; - donner l'implémentation d'une classe en conformité avec son interface et son contrat ; - utiliser correctement les exceptions pour détecter et gérer les violations de pré-conditions ; - utiliser correctement l'héritage et la généricité pour la définition de classes ; - mettre en œuvre les design patterns **Singleton**, **Enum** et **Strategy**.

1 Design patterns *Singleton* et *Enum*

Exercice 1 Un jeu vidéo nécessite un gestionnaire de scores partagé par l'ensemble des parties en cours. On souhaite garantir qu'il n'existe à tout moment qu'une **seule instance** de ce gestionnaire dans l'application. On suppose qu'une classe `Score` existe déjà, avec un constructeur `Score(String nom, int valeur)` et des méthodes `getNom()` et `getValeur()`.

- En appliquant le design pattern *Singleton* dans sa forme « classique » (constructeur privé, champ statique contenant l'unique instance, méthode statique d'accès), donnez le code source complet d'une classe `GestionnaireScores` possédant :
 - un attribut privé contenant la liste des scores enregistrés ;
 - une méthode `ajouterScore(String nom, int valeur)` qui insère un nouveau score dans la liste, **en conservant la liste triée par score décroissant** ;
 - une méthode `getTop(int n)` qui renvoie les `n` meilleurs scores (ou moins si la liste en contient moins que `n`).
- Expliquez en deux ou trois lignes pourquoi le constructeur de `GestionnaireScores` doit être `private`, et pourquoi le champ contenant l'unique instance doit être `static`.
- Récrivez entièrement la classe `GestionnaireScores` en appliquant cette fois le design pattern *Singleton* basé sur une `enum` Java (une seule constante d'énumération). Donnez le code source complet.
- En deux ou trois lignes, expliquez l'avantage principal de l'implémentation par `enum` par rapport à l'implémentation « classique », vis-à-vis de la désérialisation et de la réflexion.

IMPORTANT : pour les questions 1 et 3, les commentaires et assertions ne sont pas demandés, mais le tri par score décroissant doit être correctement assuré.

2 Design pattern *Strategy*

Exercice 2 On souhaite permettre de trier dynamiquement l'inventaire d'un joueur — une collection d'`ObjetZork` — selon un critère choisi à l'exécution. On suppose que la classe `ObjetZork` possède les méthodes `getPoids()` (renvoie un `int`) et `getNom()` (renvoie un `String`). On souhaite appliquer le design pattern *Strategy*.

1. Définissez une interface fonctionnelle `CritereDeTri` permettant de comparer deux `ObjetZork`, qui doit pouvoir être implémentée aussi bien par une classe anonyme, qu'une expression lambda, qu'une référence de méthode.
2. Donnez le code source complet d'une classe `Inventaire` contenant une `List<ObjetZork>` et possédant une méthode `trier(CritereDeTri critere)` qui trie cette liste **sur place** selon le critère fourni.
3. À l'aide d'expressions lambda, donnez deux critères de tri concrets :
 - un critère triant par poids croissant ;
 - un critère triant par ordre alphabétique du nom.
4. Une autre manière d'appliquer le pattern *Strategy* consiste à définir des classes concrètes (plutôt que des lambdas) qui implémentent l'interface. Donnez le code source complet d'une classe `TriParPoidsDecroissant` qui implémente `CritereDeTri` selon ce principe, et qui applique en plus le design pattern *Singleton* (il ne doit jamais être possible d'instancier plus d'un objet `TriParPoidsDecroissant`).

IMPORTANT : pour cet exercice, les commentaires et assertions ne sont pas demandés.

Sujet d'entraînement généré par Claude — Anthropic.